

组成原理课程第 三 次实验报告

实验名称：CPU 设计实战 Lab2

学号： 2312331 姓名： 王一诺 班次： 张金老师

一、实验内容说明

请根据《CPU 设计实战》书中的第三章讲解，完成 Lab2 的三个实验，并撰写实验报告。

1、针对任务一寄存器堆实验，完成仿真，在感想收获中思考并回答问题：为什么寄存器堆要设计成“两读一写”？

2、针对任务二同步 ram 和异步 ram 实验，可以参考实验指导手册中的存储器实验，注意同步和异步需要分开建工程，然后仿真，在感想收获中分析同步 ram 和异步 ram 各自的特点和区别。

3、针对任务三，重点介绍清楚发现 bug、修改 bug 和验证的过程，在感想收获中总结使用 vivado 调试的经验步骤。

二、实验步骤与结果分析

任务一：寄存器堆仿真实验

1.实验步骤：

- 1) 使用 vivado 新建一个工程，将 regfile_task 里的设计源码和仿真源码加入到工程中；
- 2) 对工程进行仿真测试，结合波形观察寄存器堆的读写行为。

2.代码简要分析：

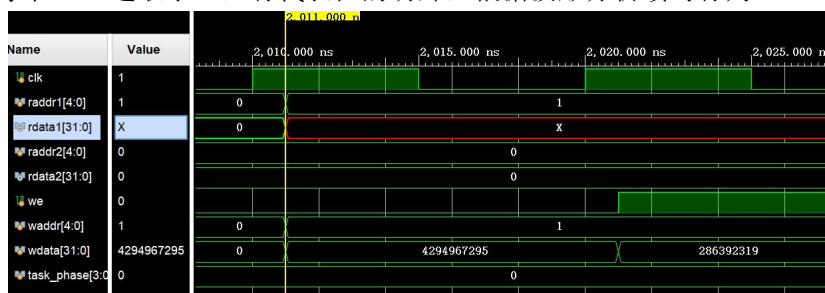
- 1) 寄存器堆模块 (regfile.v)：

端口定义：有两个读端口 (raddr1/raddr2) 和一个写端口 (waddr)，为“两读一写”结构。写操作：在时钟上升沿触发，当写使能 we=1 时，wdata 写入 waddr 指定的寄存器。读操作：组合逻辑实时输出，0 号寄存器恒为 0。

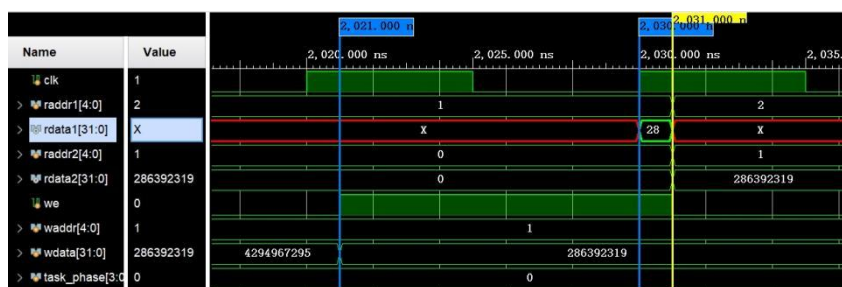
- 2) 测试平台 (rf_tb.v)：tb 分阶段测试读写行为。Part0 验证单寄存器写入与读取，以及 we 使能控制端的作用；Part1 测试连续写入多个寄存器并交叉读取；Part2 连续相邻地址读取。

3.仿真与读写行为观察：

为 tb 的每个 Part 选取了一些有代表性的切片，根据波形分析读写行为。



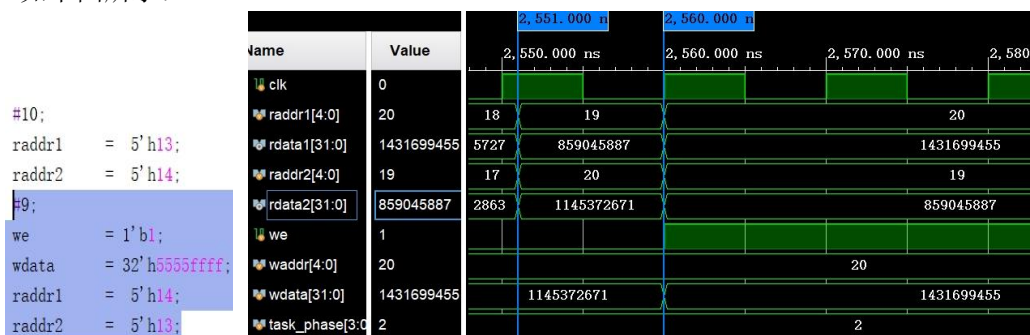
2011ns 是 tb 的第一次测试，此时 we=0 无法进行写入操作，addr=1 处未进行初始化也没有写入值，此时 raddr1 读取 1 处数值，就会返回 X 表示不定值，如上图所示。



2021ns 处 we 明明已经改为 1，为什么 addr=1 处仍然没有写入值而使 rdata 还是 X 呢？这是因为只有在时钟上升沿（且 we=1）才能对 waddr 地址处的值进行更新，所以在 2030ns 时钟上升沿且 we 还是 1 时，addr=1 处写入 wdata=286392319，rdata1 也就读出了这个值；到了 2031ns 时，we=0 锁定寄存器，读端口一读取未初始化的 2 地址处而返回 X，读端口二读取 1 地址处返回了 286392319，如上图所示。

Part1 和 Part2 部分都是连续相邻地址写入和读取，情况类似于 2021ns—2031ns 的波形，不再赘述。

为了测试读写冲突的情况，也就是当某一地址被同时读取和更新时，读取结果是旧值还是新值呢？为此我修改了 Part2 部分代码，控制对 addr=20 处在同一时钟上升沿进行读取和更新，如下图所示：



2551ns—2560ns 处读取端口 2 读的就是 20 地址处，可见此时 addr=20 的读取结果还为 1145372671；在 2560ns 处，恰为时钟上升沿且 we 改为 1，对 20 地址处改为了 1431699455，而 raddr1 的读取结果也正是 1431699455，也就是说更新和读取在同一时刻发生时，读取结果为新值。

任务二：同步 RAM 与异步 RAM 仿真、综合与实现

1. 实验步骤：

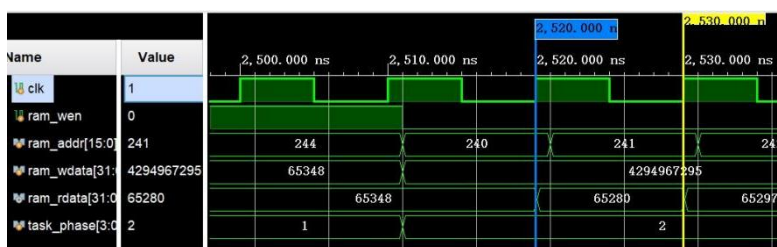
- 1) 为同步 RAM 和异步 RAM 各自建立一个项目，并把设计源码、约束文件、仿真源码加入项目，并调用 Xilinx 库 IP 生成同步/异步 RAM。
- 2) 对他们进行仿真，对比读写的异同。
- 3) 完成仿真后，对他们进行综合和实现，参考 3.3 节介绍的方法查看时序结果和资源利用率，并结合读写时序进行分析。

2. tb 代码简要分析：

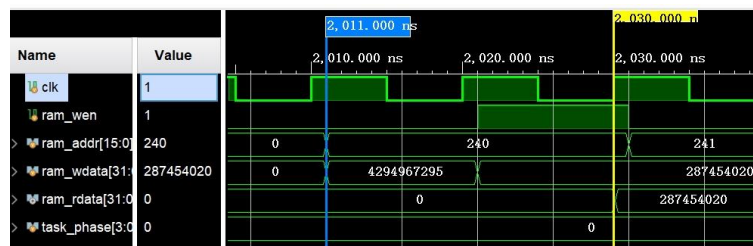
测试分为三个阶段：Part0 测试写入和读取单个地址（0xf0），先尝试无效写入，再有效写入，最后读取验证；Part1 连续写入多个地址（0xf0 到 0xf4），验证连续写入功能；Part2 关闭写使能，读取之前写入的地址，验证数据是否保持。

3. 仿真结果分析对比

1>同步 RAM:



读操作：地址变化后，需等待时钟上升沿才能捕获新数据。借助 Part2（只读不写），可以看到，地址切换时，ram_rdata 不能立刻更新，需要等到时钟上升沿才能更新读取，如上图所示。

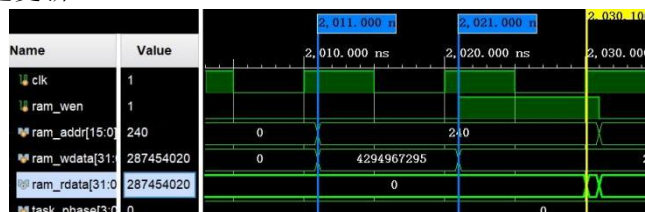


写操作：在时钟上升沿触发。如上图所示，2011ns—2021ns 间 wen=0，不能写入；2021ns 处 wen=1，但等到 2030ns 也就是时钟上升沿才触发写入和读取更新。

2>异步 RAM:



读操作：地址变化后，ram_rdata 随地址变化立即更新。如上图所示，ram_addr 变化时，ram_rdata 也立刻随之更新。



写操作：在时钟上升沿触发，但是读操作无延迟。如上图所示，2021ns 处 wen 已经变为 1，但是 rdata 没有变化，直到 2030ns 也就是时钟上升沿才变化，而从上述分析我们知道异步 ram 的读取操作是没有延迟的，也就是说写入操作在时钟上升沿触发，从而延后了更新。

4. 时序结果和资源利用率分析

(1) 资源利用率对比：

Resource	Utilization	Available	Utilization %	Resource	Utilization	Available	Utilization %
LUT	122	133800	0.09	LUT	41357	133800	30.91
FF	4	267600	0.01	LUTRAM	32768	46200	70.93
BRAM	58	365	15.89	IO	82	400	20.50
IO	82	400	20.50	BUFG	1	32	3.13
BUFG	1	32	3.13				

如上图所示，左上图为同步 RAM(Block RAM)情况，右上图为异步 RAM(Distributed RAM)。

同步 RAM 资源类型以占用 BRAM 资源为主，专用存储单元，效率高容量大；异步 RAM 资源类型以消耗查找表 LUT 和 FF 为主，LUT 资源消耗大，导致逻辑资源较紧张。

(2) 时序报告分析:

Design Timing Summary	Design Timing Summary
Setup	Setup
Worst Negative Slack (WNS): 2.439 ns	Worst Negative Slack (WNS): -3.648 ns
Total Negative Slack (TNS): 0.000 ns	Total Negative Slack (TNS): -219.196 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 232
Total Number of Endpoints: 1325	Total Number of Endpoints: 327712
All user specified timing constraints are met.	Timing constraints are not met.

如上图所示,左上图为同步 RAM(Block RAM)情况,右上图为异步 RAM(Distributed RAM)。

同步 RAM 的 WNS 为正值,说明时序裕量充足,可能是因为时钟到输出的路径延迟较稳定;异步 RAM 的 WNS 和 TNS 均为负值,但违约值不高,时序满足还是较好的,可能是因其组合逻辑延迟较长。

任务三: 数字逻辑电路的设计与调试

1.实验步骤:

1) 本实践任务提供了一个有 5 个 bug 的数字逻辑电路设计源码。该设计的正确功能是:获取开发板最右侧 4 个拨码开关的状态(记为“拨上为 1’ 拨下为 0’ ,实际开发板上拨码开关的电平是“拨上为低电平,拨下为高电平”),共有 16 个状态(数字编号是 0~15)。最左侧数码管实时显示 4 个拨码开关的状态。数码管只支持显示 0~9, 如果拨码开关状态是 10~15, 则数码管的显示状态不更改(显示上一次的显示值)。最右侧的 4 个单色 LED 灯会显示上一次的拨码开关的状态,支持显示 0~15(拨码开关拨上, 对应 LED 灯亮)。

2) 提供的设计源码中包含 5 个 bug, 其中 4 个是 3.22 节中提到的波形异常的前 4 种情况:波形为“Z”, 波形为“X”, 波形停止和越沿采样, 另外的 1 个 bug 是功能 bug。

3) 理解示例设计的功能, 分析仿真顶层 tb.v, 并理解仿真的行为。

4) 进行仿真, 并充分利用仿真的辅助小技巧进行调试, 找出所有的 bug。

2. 设计功能和仿真 tb 的理解

1>设计功能需求:

拨码开关拨上为低电平(逻辑 0), 拨下为高电平(逻辑 1), 需对输入信号 switch[3:0]取反处理; 输入状态范围为 0~15(对应拨码开关的 16 种组合); 数码管实时显示当前拨码开关状态(0~9), 若状态 ≥ 10 , 则保持上一次有效显示值, 七段数码管编码需根据输入值动态生成; LED 显示上一次拨码开关的状态(0~15), LED 灯亮表示对应位拨码开关拨上。

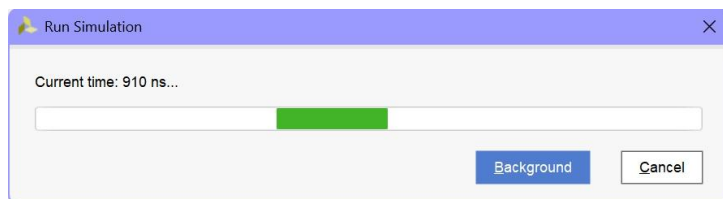
2>测试仿真(tb.v):

输入序列: 初始复位后, switch 从 0xf(逻辑 0)逐步切换为 0x8(逻辑 7)、0x9(逻辑 6)、0xe(逻辑 1)、0x2(逻辑 13)、0x0(逻辑 15)。

预期输出: 数码管依次显示 0→7→6→1→保持 1→保持 1(因 13、15 ≥ 10)。LED 初始为 0→显示 7→显示 6→显示 1→显示 13→显示 15。

3. Bug 定位与修改

(1) 波形停止



原代码进行仿真测试, 发现发生了卡死现象, 如上图所示。发生卡死很大可能是因为组合逻辑错误的形成了环路导致仿真无法收敛。检查代码, 发现了如左下图所示的逻辑环路, 这里直接移除 keep_a_g, 直接使用 num_a_g 表示超过 9 时保持旧值即可, 修改后如右下图

所示。再进行仿真，仿真正常进行，为发生卡死现象。

```
//keep unchange if show_data>=10
wire [6:0] keep_a_g;
assign keep_a_g = num_a_g + nxt_a_g;

assign nxt_a_g = show_data==4'd0 ? 7'b11111110 : //0
               show_data==4'd1 ? 7'b01100000 : //1
               show_data==4'd2 ? 7'b10110101 : //2
               show_data==4'd3 ? 7'b11110001 : //3
               show_data==4'd4 ? 7'b01100111 : //4
               show_data==4'd5 ? 7'b10110111 : //5
               show_data==4'd6 ? 7'b10111111 : //6
               show_data==4'd7 ? 7'b11100000 : //7
               show_data==4'd8 ? 7'b11111111 : //8
               show_data==4'd9 ? 7'b11110111 : //9
               keep_a_g ;

assign nxt_a_g = show_data==4'd0 ? 7'b11111110 : //0
               show_data==4'd1 ? 7'b01100000 : //1
               show_data==4'd2 ? 7'b10110101 : //2
               show_data==4'd3 ? 7'b11110001 : //3
               show_data==4'd4 ? 7'b01100111 : //4
               show_data==4'd5 ? 7'b10110111 : //5
               show_data==4'd6 ? 7'b10111111 : //6
               show_data==4'd7 ? 7'b11100000 : //7
               show_data==4'd8 ? 7'b11111111 : //8
               show_data==4'd9 ? 7'b11110111 : //9
               num_a_g ; //移除keep_a_g, 直接使用num_a_g保持旧值
```

(2) 信号为Z

Name	Value	999,992 ps	999,994 ps	999,996 ps	999,998 ps
clk	0				
resetsn	1				
switch[3:0]	2			2	
num_csn[7:0]	Z			Z	
num_a_g[6:0]	48			48	
led[3:0]	15			15	

在仿真中发现，num_csn 一直为 Z（高阻态），如上图所示。检查 num_csn 相关的定义、实例化和赋值等代码后发现，show_sw 模块实例化 shou_num 时，端口 num_csn 被错误命名为 num_scn 导致悬空。直接将命名修改正确即可，修改后再进行测试，num_csn 正常显示有效值。

(3) 越沿采样

Name	Value	200,000 ns	400,000 ns	600,000 ns	800,000 ns	1,000,000 ns
clk	X					
resetsn	X					
switch[3:0]	XXXX					
num_csn[7:0]	01111111			01111111		
num_a_g[6:0]	XXXXXXXX					
led[3:0]	XXXX					

观察波形，发现 led 一直保持初始状态不变，猜测原因可能是赋值与更新出错误，或者，还有一种可能——思考 led 的更新逻辑，它是在 show_data_r != show_data 也就是显示数值更新时才会随之更新，将上一个状态的拨码开关状态 prev_data 记录下来。那么，如果阻塞赋值导致时序错位，使得 show_data_r != show_data 一直为假，prev_data 迟迟不更新，led 也就不会更新。检查原代码后发现果然如此，如左下图所示。这属于越沿采样 bug。

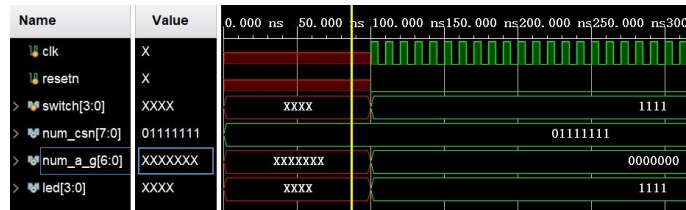
```
always @(posedge clk)
begin
    show_data_r = show_data;
end

always @(posedge clk)
begin
    show_data_r <= show_data;
end
```

修改后如右上图所示。再进行仿真，led 正常更新，如下图所示：

Name	Value	200,000 ns	400,000 ns	600,000 ns	800,000 ns
clk	1				
resetsn	1				
switch[3:0]	0010				
num_csn[7:0]	01111111				
num_a_g[6:0]	01100000				
led[3:0]	1110				

(4) 信号为X



100ns 前多个数值为 x，这是初始化问题，完善初始化代码即可。

(5) 功能错误

没有发现功能性错误，按照以上四处修改完毕后，num_a_g 在 resetn 置 1 后，短暂的显示 0，然后依次对应显示 7→6→1→保持 1→保持 1，最后两个大于 9 的数保持上一个状态也就是 1 的功能也正常实现了，没有发现错误；led 也按照预期数值实现了，没有发现错误。

三、总结感想

1. 为什么寄存器堆要设计成“两读一写”？

RISC 架构的指令集中，大多数算术逻辑指令需要同时读取两个寄存器，并将结果写入目标寄存器。采用两读一写的设计，单周期内完成双操作数读取和结果写入，可以提高指令吞吐率，支持流水线高效运行。此外，匹配典型指令格式，简化控制逻辑设计。

2. 同步 ram 和异步 ram 各自的特点和区别？

同步 RAM 依赖时钟同步，时序可控性强；使用专用存储单元，资源利用率高；简化了时序控制，但牺牲了即时响应能力。

异步 RAM 依赖组合逻辑，时序易受路径延迟影响；读操作响应快，但是消耗大量 LUT 资源，限制整体设计规模；可以说是以资源开销为代价换取了快速访问，适用于小规模且需要低延迟的场景。

3. vivado 调试的经验

调试就是一个发现 bug 和错误、修复 bug 和错误的过程，所以首先，要有预期值；随后，观察仿真结果，依据学到的四种常见 bug 知识定位出现 bug 的错误代码，根据预期值与实际值的不同定位出现功能性 bug 的错误代码；最后，修复代码，再进行仿真调试。

Vivado 的调试与 C++ 和 py 很不一样，比起用断点调试观察输出来修改错误，vivado 更需要对设计整体的理解。模块与模块之间通常相互关联，只有在整体设计中定位模块，才能更快地从错误波形定位到错误代码，完成修改。